Лабораторная работа №9
Работа с текстовыми файлами

1 Цель работы

1.1 Научиться работать с текстовыми файлами средствами языка C#.

2 Литература

2.1 Ашарина, И. В. Объектно-ориентированное программирование в C++: лекции и упражнения. Учебное пособие для вузов. – 2-е изд., перераб. и доп. Москва : Горячая Линия–Телеком, 2017. – URL: https://ibooks.ru/bookshelf/359752/reading. – Режим доступа: для зарегистрир. пользователей. – Текст : электронный. – п.9.2-9.3.

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

При реализации программ воспользоваться разными способами работы с текстовыми файлами – через статические методы File и файловые потоки со StreamReader/StreamWriter

5.1 Написать программу, запрашивающую у пользователя имя файла. Если файла с указанным именем не существует, сообщить об этом пользователю.

5.2 Изменить приложение из п.5.1, чтобы пользователь после ввода существующего имени файла увидел на экране его содержимое, считанное построчно из файла и выведенное на экран (считать, что в файле содержатся строковые данные).

5.3 Написать программу, перезаписывающую данные из первого файла во второй файл, имена файлов указывает пользователь с клавиатуры.

5.4 Написать программу, запрашивающую у пользователя имя файла и открывающего файл для дозаписи в конец: сначала выводит содержимое, а после позволяет вводить строки до тех пор, пока не будет введена строка /end (она не должна быть записана в файл).

5.5 Написать программу, считывающую из файла данные посимвольно. Подсчитать число вхождений в файл указанного пользователем символа. Результат вывести на экран.

6 Порядок выполнения работы

6.1 Используя Microsoft Visual Studio, создать проект C# и выполнить

задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Как открыть файл для добавления информации в C#?

8.2 Каков порядок закрытия файла в C#?

8.3 Как открыть файл для чтения информации в C#?

9 Приложение

9.1 Общие сведения о работе с файлами

Файл — это именованная область хранения данных на внешнем носителе информации. Работа с файлами позволяет программе сохранять данные между запусками, считывать ранее записанную информацию и изменять её.

В языке C# для работы с файловой системой используется пространство имён System.IO, предоставляющее классы для создания, чтения, записи, копирования и удаления файлов.

Для его подключения используется директива:

```
using System.IO;
```

9.2 Пути к файлам

Для обращения к файлу необходимо указать его расположение. Путь к файлу может быть абсолютным или относительным.

Абсолютный путь содержит полное расположение файла, начиная с корневого каталога файловой системы.

Пример:

```
string path = @"C:\Data\example.txt";
```

Символ @ перед строковым литералом позволяет записывать обратные косые черты \ без дополнительного экранирования.

Относительный путь задаёт расположение файла относительно текущего рабочего каталога программы.

Пример:

```
string path = "example.txt";
```

В этом случае файл будет искаться в текущем рабочем каталоге.

Для определения текущего рабочего каталога можно использовать:

```
Console.WriteLine(Directory.GetCurrentDirectory());
```

Следует учитывать, что рабочий каталог не всегда совпадает с каталогом, в котором находится исходный код программы, или с каталогом исполняемого файла.

Для объединения частей пути рекомендуется использовать метод Path.Combine():

```
string folder = "Data";
string fileName = "notes.txt";

string path = Path.Combine(folder, fileName);
```

Метод самостоятельно использует разделители каталогов, соответствующие операционной системе.

9.3 Работа с файлами с помощью класса File

Класс File предоставляет статические методы для выполнения операций над файлами. Его методы вызываются непосредственно через имя класса, поэтому создавать экземпляр File не требуется.

9.3.1 Проверка существования файла

Перед чтением файла полезно убедиться в том, что он существует.

Для этого применяется метод:

```
File.Exists(path);
```

Метод возвращает значение типа bool:

- true – файл существует;
- false – файл не найден либо невозможно подтвердить его существование.

Пример проверки:

```
string path = "notes.txt";

if (File.Exists(path))
{
    Console.WriteLine("Файл найден");
}
else
{
    Console.WriteLine("Файл не найден");
}
```

9.3.2 Чтение содержимого файла

Класс File предоставляет несколько способов чтения текстовых данных.

Метод	**Назначение**
File.ReadAllText(path)	Считывает весь текст файла в одну строку
File.ReadAllLines(path)	Считывает все строки файла в массив строк
File.ReadLines(path)	Позволяет последовательно перебрать строки файла

Пример получения всего содержимого:

```
string text = File.ReadAllText("notes.txt");
```

При таком подходе всё содержимое файла загружается в память.

Для получения массива строк используется:

```
string[] lines = File.ReadAllLines("notes.txt");
```

При обработке больших файлов более удобным может оказаться File.ReadLines(), позволяющий перебирать строки без предварительной загрузки всего файла:

```
foreach (string line in File.ReadLines("notes.txt"))
{
    // Обработка очередной строки
}
```

9.3.3 Запись данных в файл

Для записи текстовых данных используются следующие методы:

Метод	**Назначение**
File.WriteAllText(path, text)	Записывает текст, заменяя прежнее содержимое
File.WriteAllLines(path, lines)	Записывает коллекцию строк
File.AppendAllText(path, text)	Добавляет текст в конец файла
File.AppendAllLines(path, lines)	Добавляет коллекцию строк в конец файла

Пример записи:

```
File.WriteAllText("message.txt", "Пример текста");
```

Если файл уже существует, метод WriteAllText() заменяет его содержимое. Если файла нет, он будет создан при наличии необходимых прав.

Для добавления новых данных без удаления ранее записанной информации применяется:

```
File.AppendAllText("message.txt", "Новые данные");
```

Обратите внимание: метод AppendAllText() не добавляет символ перевода строки автоматически. Если требуется начать запись с новой строки, необходимо самостоятельно предусмотреть разделитель, например Environment.NewLine.

9.4 Работа с потоками StreamReader и StreamWriter

Поток представляет собой абстракцию последовательного чтения или записи данных.

Классы StreamReader и StreamWriter предназначены для работы с текстовыми данными и обеспечивают доступ к содержимому файла через объект потока.

9.4.1 Чтение с помощью StreamReader

Для открытия файла на чтение можно создать объект класса StreamReader:

```
StreamReader reader = new StreamReader("notes.txt");
```

После открытия файла становятся доступны методы чтения.

Метод или свойство	**Назначение**
Read()	Считывает следующий символ
ReadLine()	Считывает одну строку
ReadToEnd()	Считывает весь оставшийся текст
EndOfStream	Показывает, достигнут ли конец потока
Close()	Закрывает объект чтения

Метод ReadLine() возвращает строку типа string, а при достижении конца файла — null. Символы окончания строки в возвращаемый результат не включаются.

Пример:

```
string? line = reader.ReadLine();
```

Для последовательного чтения можно использовать цикл:

```
while (!reader.EndOfStream)
{
    string? line = reader.ReadLine();

    // Обработка считанной строки
}
```

После завершения работы поток необходимо закрыть.

9.4.2 Посимвольное чтение

Для чтения файла по одному символу применяется метод Read().

```
int value = reader.Read();
```

Метод возвращает значение типа int, представляющее код считанной единицы текста UTF-16, либо -1, если достигнут конец файла.

Поэтому возвращаемое значение сначала следует сравнить с -1, а затем при необходимости преобразовать в char:

```
if (value != -1)
{
    char symbol = (char)value;
}
```

Тип char представляет одну кодовую единицу UTF-16. Большинство обычных букв, цифр и знаков занимают одну такую единицу, но некоторые символы Unicode, например эмодзи, состоят из двух.

Для последовательного чтения данных обычно применяется цикл, выполняющийся до достижения конца файла.

9.4.3 Запись с помощью StreamWriter

Класс StreamWriter предназначен для записи текстовых данных.

Пример создания объекта:

```
StreamWriter writer = new StreamWriter("output.txt");
```

При таком способе открытия существующее содержимое файла будет удалено.

Для добавления данных в конец существующего файла необходимо передать вторым аргументом значение true:

```
StreamWriter writer = new StreamWriter("output.txt",
true);
```

Таблица 3 – Основные методы StreamWriter

Метод	**Назначение**
Write()	Записывает данные без автоматического перевода строки
WriteLine()	Записывает данные с переводом строки
Flush()	Сбрасывает буфер записи в базовый поток
Close()	Закрывает объект записи

Пример:

```
writer.Write("Первая часть. ");
writer.WriteLine("Вторая часть.");
```

Метод WriteLine() добавляет символы окончания строки после записанного текста.

9.5 Управление временем жизни файловых потоков

При открытии файлового потока приложение получает системные ресурсы, которые необходимо своевременно освобождать.

Если не закрыть поток, часть записанных данных может остаться в буфере, а файл может продолжать удерживаться открытым.

Для автоматического освобождения ресурсов рекомендуется использовать конструкцию using.

Пример:

```
using (StreamReader reader = new StreamReader("notes.txt"))
{
    // Операции чтения
}
```

После выхода из блока using вызывается метод Dispose(), освобождающий связанные с потоком ресурсы.

Также в современных версиях C# допускается сокращённая форма:

```
using StreamReader reader = new StreamReader("notes.txt");
```

В этом случае объект будет освобождён при завершении области видимости, в которой объявлена переменная.

Конструкция using рекомендуется вместо ручного вызова Close(), поскольку освобождение ресурсов происходит даже при возникновении исключения внутри блока.

9.6 Обработка ошибок при работе с файлами

Операции с файлами могут завершаться ошибками по причинам, не зависящим от программы: отсутствие файла, неправильный путь, ограничение доступа, ошибки ввода-вывода.

Для обработки подобных ситуаций используется конструкция try-catch.

Общий синтаксис:

```
try
{
    // Операции, при выполнении которых
    // могут возникнуть исключения
}
catch (IOException ex)
{
    Console.WriteLine($"Ошибка ввода-вывода: {ex.Message}");
}
```